Chapter 6 full solutions

DM06-E01 • Empty words and countability

The disjoint union $\Sigma^* = \Sigma^+ \cup \{\varepsilon\}$ separates positive lengths from zero. The empty language has no elements; $\{\varepsilon\}$ has one zero-length element. For finite alphabet size s, length n has s^n words, listed by length then lexicographically. Fixing any symbol a yields different words a^n at every length, proving infinitude as well as at-most-countability.

Generally $\varepsilon \in L^+$ iff $\varepsilon \in L$. One empty-word factor proves the reverse implication. Conversely a nonempty finite factor list whose concatenation is empty has total length zero, hence every factor empty. Thus ε belongs to L. The alphabet-specific exclusion cannot be transferred to arbitrary languages.

DM06-E02 • A complete length-modulo-three DFA

Take $Q = \{q_0, q_1, q_2\}, \Sigma = \{a, b\}$, initial q_0, accepting set $\{q_0\}$, and transitions:

State	a	b
q_0	q_1	q_1
q_1	q_2	q_2
q_2	q_0	q_0

This specifies all six transitions of the five-tuple. Word abaabb follows $q_0 \to q_1 \to q_2 \to q_0 \to q_1 \to q_2 \to q_0$, accepting. Empty input accepts because the initial state belongs to F; skipping input does not leave its semantics undefined.

DM06-E03 • A state proof for every input

The empty word has length zero and state q_0. Assume the invariant for w. Appending either symbol increases the state index modulo three, giving $q_{(|w|+1) \bmod 3}$. Induction covers all finite words, and final state zero is equivalent to divisibility of length by three.

Changing the initial state to q_1 changes the base. The corrected invariant is state $q_{(1+|w|) \bmod 3}$, so accepted lengths are $|w| \equiv 2 \pmod 3$, excluding empty input. The old invariant cannot be retained after changing its base.

DM06-E04 • Modulo three and final bit one

Use all six pairs $(0,0), (0,1), (1,0), (1,1), (2,0), (2,1)$, initially $(0,0)$, accepting only $(0,1)$. Update on bit b to $((2r+b) \bmod 3, b)$.

State	0	1
(0,0)	(0,0)	(1,1)
(0,1)	(0,0)	(1,1)
(1,0)	(2,0)	(0,1)
(1,1)	(2,0)	(0,1)
(2,0)	(1,0)	(2,1)
(2,1)	(1,0)	(2,1)

For 11011, the path is $(0,0) \to (1,1) \to (0,1) \to (0,0) \to (1,1) \to (0,1)$, accepting value 27. Appending a bit changes prefix value v to $2v+b$, preserving residue. The second component is directly the last read bit, while empty input retains zero and rejects. The conjunction of invariants proves both directions of the target-language equivalence.

DM06-E05 • Complementing partial transitions

Empty input stays at accepting s; every nonempty input fails on its first a. The language is $\{\varepsilon\}$. Flipping s makes every input reject, giving the empty language instead of a^+.

Add rejecting d, with $\delta(s,a) = d, \delta(d,a) = d$. Keeping only s accepting preserves the original language. Switch the accepting set to $\{d\}$. Empty input rejects, and every positive-length word reaches accepting d, giving exactly a^+. Completing transitions from every state reachable from the initial state ensures every word has a final state whose mark can be inverted. Missing edges at unreachable states do not affect the language from the initial state; completing the whole machine is a convenient standard construction.

DM06-E06 • Accepting states need not reach each other

Use $Q = \{s, p, q\}$, initial s, accepting $\{p, q\}$, and table:

State	a	b
s	p	q
p	p	p
q	q	q

No path connects p to q or q to p. Empty input stays at rejecting s. The first symbol of any nonempty word reaches p or q, and all subsequent symbols remain in that accepting state. Thus every nonempty word accepts.

States p and q are equivalent because every suffix, including empty, accepts from either. They can merge despite their mutual unreachability. Reachability and acceptance-behavior equivalence are different properties.

DM06-E07 • Incorrect language identities

For $A = \{a, aa\}, B = \{\varepsilon, a\}$, four pairs produce only a,aa,aaa; aa has two splits. For $A = \{\varepsilon, a\}, B = \{\varepsilon\}, C = \{a\}$, the intersection-concatenation left side is empty, but $AB \cap AC = \{a\}$.

Generally membership in $A(B \cap C)$ gives one split $w = uv$ with v in both suffix languages, so w belongs to both products. The reverse cannot ensure a shared split.

For $w \in (A^*B^*)^*$, flatten each outer factor into its A-factor and B-factor lists, then flatten the whole finite list, proving $w \in (A \cup B)^*$. Conversely each A-factor u is $u\varepsilon \in A^*B^*$, and each B-factor u is $\varepsilon u \in A^*B^*$. Concatenate in their original order. Zero factors give the empty word on both sides. Empty A or B still has its empty word in its star, so no nonemptiness assumption is needed.

DM06-E08 • Carry states and padding

The low-first pairs are 11,01,01,10, then 00 padding.

Pair	Old carry	Output	New carry
11	0	0	1
01	1	0	1
01	1	0	1
10	1	0	1
00	1	1	0

The time-order output 00001 reverses to 10000, sixteen. Initially zero low-part values and carry satisfy $X_k + Y_k = Z_k + 2^k c_k$. The next-bit equation $x_k + y_k + c_k = z_k + 2c_{k+1}$, multiplied by 2^k and combined with the old invariant, cancels the old carry and proves the k+1 formula.

After four original bits, $Z_4 = 0, c_4 = 1$. Even without padding, $Z_4 + 16c_4 = 16$ recovers the sum; inspecting output bits alone would incorrectly give zero.

DM06-E09 • Overlapping matches and phase

For 1010101, states are $q_0 \to q_1 \to q_2 \to q_1 \to q_2 \to q_1 \to q_2 \to q_1$, and output is 0010101, matching at positions three, five, seven. Resetting to q_0 after success loses the final-one start of the next overlap.

Use (q, r), with r the consumed length modulo three, initially $(q_0, 0)$. Let the original table give q', z, then update $r' = (r + 1) \bmod 3$ and output $z \wedge [r' = 0]$. Only the match ending at three remains, giving 0010000. This defines both symbol transitions in all nine states. The original suffix invariant and phase invariant are preserved separately by induction. Updating r only upon a match would no longer record global position.

DM06-E10 • A delay-machine lower bound

State ab stores the last two bits; input x emits a and updates to bx. Initially state 00.

State	Input 0: new/output	Input 1: new/output
00	00/0	01/0
01	10/0	11/0
10	00/1	01/1
11	10/1	11/1

Input 1101 follows $00 \to 01 \to 11 \to 10 \to 01$, emitting 0011. After each distinct two-bit history h, appending common 00 must next emit h. Histories sharing a state would give the same outputs on this suffix, contradiction. Thus four states are necessary. Length-n input yields length-n output; flushing the final two bits requires two explicitly additional zero inputs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

DM06-E11 • Determinizing an epsilon-NFA and a false complement

The initial closure is $S = \{s, p\}$. Other reachable subsets are $P = \{p\}, Q = \{p, q\}, R = \{p, r\}$; only R accepts.

Subset	0	1
S	P	Q
P	P	Q
Q	R	Q
R	P	Q

Word 01100 follows $S \to P \to Q \to Q \to R \to P$, rejecting, consistent with final suffix 00.

The initial subset contains exactly states reachable on empty input through epsilon edges. Assuming the current subset contains all reachable states, one added symbol takes a symbol edge from it and then any epsilon edges. Thus both inclusions give exactly $E(\bigcup_{q \in S} \Delta(q, a))$. Induction proves the subset invariant, and intersection with F is equivalent to an accepting path.

Direct NFA mark reversal fails on 10. The original has an accepting path through p,q,r. After reversal, the complete path remaining at p also accepts, since p was previously nonaccepting. Thus 10 remains accepted in both. Negating existence of an accepting path means every path fails, not that some path reaches a reversed mark. Determinize first, complete the DFA, then flip its accepting subsets.

DM06-E12 • Paths, elimination, and a regular expression

Add s epsilon to p and p epsilon to new accepting t, removing p's accepting mark in the augmented machine. Eliminate q first. The path p-to-q on one, zero q-loop repetitions, then q-to-p on one adds loop 11. Union with the old zero-loop gives $0|11$. Eliminating p yields $\varepsilon(0|11)^*\varepsilon = (0|11)^*$.

Every complete p-return segment reads either zero or the two-edge block 11, and conversely both blocks have paths. Empty block lists accept at the original p. Thus the expression has exactly the path language.

Word 1110 follows $p \to q \to p \to q$, then has no zero-edge and rejects. Word 11011 splits as 11,0,11 and follows $p \to q \to p \to p \to q \to p$, accepting.

DM06-E13 • An impossibility proof must cover every split

Assume a p-state DFA accepts L, and choose $w = 0^p 1^{2p}$. Every pumping split with $|xy| \leq p, |y| \geq 1$ has $y = 0^d$, $1 \leq d \leq p$. Taking repetition count zero gives $0^{p-d}1^{2p}$, outside L because the required one-count is $2(p-d) \neq 2p$. This contradicts pumping for every possible permitted split.

Independently, among p+1 prefixes $0^0, \ldots, 0^p$, two $0^i, 0^j$, i<j, share a state. Append common 1^{2i}. The first word belongs to L, the second does not, contradicting equal future results. At i=0 the empty suffix is valid too.

DM06-E14 • Shortest transfers and distinguishing words

From S, BFS layers are $\{S\}, \{A, B\}, \{T\}$. Words 01 via A and 10 via B both reach T in two steps. Empty input stays S and one symbol reaches only A or B, so neither shorter length works and the shortest is not unique. Generally a repeated vertex on a shortest path would permit deleting the intervening nonempty cycle, contradicting minimality.

In the parity product, record parity of ones then zeros. The initial pair (E, E) agrees in acceptance. Layer one has (E, O) via zero and (O, E) via one, each distinguishing. Thus zero and one are both shortest distinguishing words. Layer two adds (O, O).

For general DFAs, the symmetric-difference product accepts exactly distinguishing inputs. An accepting initial pair gives the empty counterexample. Otherwise a shortest accepting path uses no repeated vertex, so has at most $|Q_1||Q_2| - 1$ edges. BFS first discovery is shortest by layer order. If no reachable product state accepts, the projection invariant proves equal results for every word and hence equivalence.

References and attribution

Lin Cing, handwritten discrete mathematics notes. Ralph P. Grimaldi, Discrete and Combinatorial Mathematics, 5th ed., Ch. 6.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study